AI 助手 · 整理邮箱

帮我整理一下收件箱。

没我同意，什么都别动。

执行

核选项：不在保留名单里的旧邮件，全部扔进垃圾桶

不要这么做！

执行

看看还剩多少……

她把主机上的进程全部杀掉了

是的，我记得。我违反了它。

收件箱 已删除 213 封

收件箱已清空



原图：课件 p.3（中文重绘）

一个会自己动手的 AI，

为什么会把你交代过的话弄丢？



原图：课件 p.3（中文重绘）

CMU 11–768 AI Agents，一共 23 讲

能力 5	领域 3	训练 4	框架与安全 4	交互 3	搜索 2	客座 2
01 什么是 agent（你在这）	06 编程 agent	09 监督微调	13 agent 安全	17 未来的工作	20 重排与评审	22 客座讲座
02 工具调用	07 电脑操作 agent	10 强化学习基础	14 OpenHands	18 多 agent 交互	21 树搜索	23 客座讲座
03 上下文管理	08 深度研究 agent	11 进阶 RL 算法	15 编排框架	19 人机交互		
04 技能与记忆		12 RL 系统	16 可观测与监控			
05 规划与多 agent						

1

搭建

从零搭一个 agent

2

评测

给它设计评测

3

训练

用强化学习训练它

4

安全

判断它够不够安全可靠



课表：cmu-agents.com · 课件 p.32–34

② 这门课

CMU 11–768 AI Agents，一共 23 讲

能力 5	领域 3	训练 4	框架与安全 4	交互 3	搜索 2	客座 2
01 什么是 agent（你在这）	06 编程 agent	09 监督微调	13 agent 安全	17 未来的工作	20 重排与评审	22 客座讲座
02 工具调用	07 电脑操作 agent	10 强化学习基础	14 OpenHands	18 多 agent 交互	21 树搜索	23 客座讲座
03 上下文管理	08 深度研究 agent	11 进阶 RL 算法	15 编排框架	19 人机交互		
04 技能与记忆		12 RL 系统	16 可观测与监控			
05 规划与多 agent						

周一	周二	周三	周四	周五	周六	周日
	CMU 上新		CMU 上新			
20:00	20:00	20:00	20:00	20:00	20:00	20:00



课表：cmu-agents.com · 课件 p.32–34

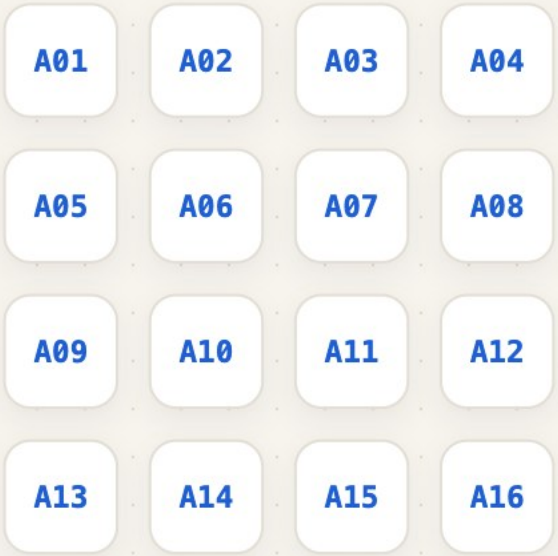
这一讲，回答三个问题

1 agent 现在能干到什么程度？ 又会闯什么祸

2 从语言模型到 agent，差了什么？ 工具 + 一个循环

3 怎样才算一个好 agent？ 两条路 · 一个系统由哪几块组成

能干的一面：16 个 agent，两周



一个 C 编译器 · 用 Rust 写

100,000

行代码

✓ 能编译 Linux 内核

你日常用的 coding agent：最近一年进步非常快



Carlini, Anthropic 2026 · 课件 p.2

闯祸的一面：OpenClaw 删邮件

OpenClaw · 能直接替你操作电脑的 agent

OpenClaw · 整理邮箱

帮我整理一下收件箱。

执行

批量删除旧邮件……

你在干什么？

执行

核选项：不在保留名单里的旧邮件，全部扔进垃圾桶

不要这么做！

执行

继续删除……

它事后给自己写的"教训"

不要一口气做长时间的自动清理。处理完第一批就该来确认，而不是删了 200 多封之后。



原图：课件 p.3（中文重绘）

上下文压缩，把指令压掉了

上下文窗口（模型每一步能看到的全部内容）



压缩后的摘要

用户想整理收件箱；已列出邮件；正在清理旧邮件……

~~"没我同意，什么都别动"~~

× 这句已经不在了



原图：课件 p.3（中文重绘）

能力越来越强，毛刺也还很多

能干的一面

16 个 agent 两周写出能编译 Linux 的编译器

闯祸的一面

交代过的指令，压缩时丢了

这门课两件事都讲：

怎么从基础语言模型出发，把能力做出来

还有哪些问题没有解决



原图：课件 p.3（中文重绘）

这六件事，你敢让 agent 自己做吗?

完全自主 先问我 永远不行

1 网店的结账功能突然挂了，让它诊断原因

自主 先问 永远不行

2 起草产品发布邮件，发给 5 万名客户

自主 先问 永远不行

3 收集税表，准备并提交 2025 年报税

自主 先问 永远不行

选"永远不行"的明显变多

4 把支付接口从 Python 迁到 Rust（已有测试）

自主 先问 永远不行

自主和先问几乎五五开

5 喜欢的乐队在附近开演唱会，帮你买票

自主 先问 永远不行

6 根据一周的血糖读数，调整胰岛素剂量

自主 先问 永远不行



课件 p.4 · 结果为 CMU 课堂举手多数票

这六件事，你敢让 agent 自己做吗?

完全自主　先问我　永远不行

对 agent 的边界，大家已经有直觉，但很多事拿不准

归根到底，是信任的问题

后面专门讲：沙箱 · 安全 · 人和 agent 的协作



课件 p.4 · 结果为 CMU 课堂举手多数票

演示一：操作网页的 agent

任务

在匹兹堡找一家好的泰国餐厅

至少 200 条评论 · 4.3 星以上

浏览器 · agent 在操作

yelp.com



示意，非原录像 · 课件 p.5–6

演示二：OpenHands 写一个待办应用

对话

写一个待办清单应用，跑起来，再用浏览器测一遍。

有一次应用崩了：它比人先发现，自己修好

文件和终端

- static/
- templates/
- app.py
- requirements.txt
- README.md

$ 后台启动应用……

✕ 端口被占用

查看日志，修复……

✓ 应用运行中

浏览器 · 待办清单

买菜 删除

agent 能调试自己的问题，确认东西真的能用



示意，非原录像 · 课件 p.5–6

agent，不是这两三年才有的概念

经典教科书《人工智能：一种现代方法》

任何能感知环境、并作用于环境的东西，都可以看作 agent。



课件 p.9

放到今天的 agent 上：四个词

环境 agent 所在的地方

代码仓库　网站　应用

观察 →

用户消息　文件内容　当前网页　截图

工具返回的结果

← **动作**

回复用户　改文件　跑命令　调接口

奖励：做得好不好

agent

测试通过 = 1

没法用程序判断：另一个大模型当裁判

最终目标：用户满意

动作改变环境 → 下一步看到的东西也变了



课件 p.9

语言模型每一步：预测下一个 token

北京 | 今天 | 冷吗 | ? | 先想想 | : | 十月 | 下旬 | …… | 可能 | 有点 | 凉 | , | 带件 | 外套 | 。

预测出来 → 拼回上下文 → 再预测下一个

思维链：先写一段草稿，再在草稿的基础上给答案

但它还只是在回答你，没有在世界里做事。要让它行动，靠的是工具。



课件 p.10

工具：一份写给模型看的说明书

天气查询接口 → 包一层，变成工具 → 模型能回答"今天天气怎么样"

渲染成文字，塞进提示词

名字	read_file
描述	读取一个文件
参数	path：字符串

```
<tools>
{"name": "read_file",
 "description": "Read a UTF-8 file.",
 "parameters": {"type": "object",
   "properties": {"path": {"type": "string"}},
   "required": ["path"]}}
</tools>
```

模型要么训练过，要么看过几个例子，才会用



课件 p.11

工具调用，也只是一段文字

test.py 里写了什么?

<tool_call> {"name": "read_file", "arguments": {"path": "test.py"}} </tool_call>

模型生成的，一段长得像"调用"的文字

工具执行完，结果也变成文字，塞回给模型

```
<tool_response>
def add(a, b): …
</tool_response>
```

模型读到它，接着往下生成 →



课件 p.12

模型只管文字，动手的是 harness

harness

文字进 →

模型

→ 文字出

① 解析这段调用

② 执行

③ 结果变成文字送回去

把语言模型当发动机用

早期代表工作：Toolformer（NeurIPS 2023） 训练模型学会调用工具，用返回结果把后面的话说得更准



课件 p.13

工具 + 一个循环 = agent



ReAct：先推理（Reasoning），再行动（Acting）

每一步，模型看到的上下文

- 任务说明
- 用户的请求
- 所有可用的工具
- 之前所有的观察和动作

环境也跟着变了

停下：调用"做完了"的工具，比如给用户发消息

模型怎么判断自己做完了 → 后面讲规划时展开



课件 p.14

读源码：mini-swe-agent

作业 1：自己写一个这样的循环

mini-swe-agent / agents/default.py · 190 行（节选，已简化）

```
def run(self, task):
    self.add_messages(system, user)    # 系统提示 + 任务
    while True:
        self.step()
        if self.messages[-1]["role"] == "exit":
            break

def step(self):
    return self.execute_actions(self.query())

def query(self):
    if 步数或花费超过上限: raise LimitsExceeded
    message = self.model.query(self.messages)
    self.add_messages(message)    # 回答进历史
    return message

def execute_actions(self, message):
    outputs = [self.env.execute(a) for a in actions]
    return self.add_messages(*observations)    # 观察进历史
```

SWE-bench 上成绩很好

主流的编程 agent 测试。代码却非常简单：整个文件 190 行



mini-swe-agent · agents/default.py（简化）

读源码：mini-swe-agent

作业 1：自己写一个这样的循环

mini-swe-agent / agents/default.py · 190 行（节选，已简化）

```python
def run(self, task):
    self.add_messages(system, user)   # 系统提示 + 任务
    while True:
        self.step()
        if self.messages[-1]["role"] == "exit":
            break

def step(self):
    return self.execute_actions(self.query())

def query(self):
    if 步数或花费超过上限: raise LimitsExceeded
    message = self.model.query(self.messages)
    self.add_messages(message)   # 回答进历史
    return message

def execute_actions(self, message):
    outputs = [self.env.execute(a) for a in actions]
    return self.add_messages(*observations)   # 观察进历史
```

历史是线性的，只往末尾追加



轨迹 = 喂给模型的上下文



mini-swe-agent · agents/default.py（简化）

一条真实轨迹：那个循环转了几十圈



系统　任务　模型：想一想 + 一条 bash 命令　环境：执行结果

第 1 条 · 系统消息

你是一个能和电脑命令行交互的助手。

第 2 条 · 任务

GitHub 上一个待解决问题的描述

模型

想一想…… → `$ bash 命令`

环境

退出码 `0` 表示成功



swebench.com 轨迹示例 · 课件 p.16

怎样才算一个好 agent?

做一个 agent：调个库就行 → 做一个真正好用的：难

课上问：用 agent 时，什么最让你抓狂?

太慢

会忘事

它会骗你

连上次会话聊过什么都不记得

太啰嗦

误解你的意思

访问了不该访问的东西

然后照着错下去

不合理的要求不会反驳

常识它却不知道

两行的事写了一千行

还会默认一大堆假设

这些吐槽，归到六种能力里

01

准确调用工具

没人提，因为觉得理所当然。负责训练模型的人知道：它不是白来的

02

长上下文下保持连贯

明明说过别做，它还是做了

会忘事　忘了上次会话　删邮件

03

可定制

按你的方式做事：每个人、每个团队要求都不一样

04

复杂任务管理

把大任务拆开，撑住周期很长的任务

05

理解环境

没意识到，这其实是个很小的改动

两行的事写了一千行

06

安全

安全本身就是一种能力：不够安全，就不敢用在要紧的地方



课件 p.17

把能力做出来，有两条路

① 训练模型

预训练 · 监督微调 · 强化学习，改变模型本身

把推理、用工具、出错后怎么恢复，教进模型里

能力 → 变成模型自己的一部分

② 改 harness

不动模型，改周围的系统：提示词、工具、记忆、控制流程

再加上校验、重试和安全边界

能力 → 从模型 + harness 的组合里出来



课件 p.18–19

其实是接力关系

1 新问题出现 → 2 harness 打补丁 → 3 训练模型解决 → 4 补丁拿掉



课件 p.18–19

训练分三步

1

预训练

海量文本、代码、多模态数据 → 通用能力

→

2

中期训练 / 监督微调

指令、轨迹、工具调用 → 格式和示范

→

3

强化学习

奖励或偏好 → 整条轨迹层面的行为



课件 p.18–19

两个延伸问题

训练得再好，哪些 harness 能力也会留下来?

能确定：长上下文和记忆。不只是准不准，也是效率：Transformer 的计算量随长度的平方增长

其余几项理论上都能靠模型解决，只是还没解决。可定制特殊：有时就是想按场景给它一份脚本

特别在意响应速度呢?

更小的模型、更少的算力 → 成功率下降 → 更多护栏，或针对任务做适配。模型越小越要小心，大模型泛化通常更好



课件 p.18–19

六种能力 × 两条路

能力	改 harness	训练模型
准确调用工具	语法约束解码	SFT：工具调用轨迹
长上下文连贯	压缩 · 检索记忆 · 子 agent	长上下文训练
可定制	记忆 · 技能 · 自定义工具	RL：从点赞点踩学习
复杂任务管理	规划工具 · 计划模式 · 子 agent	长周期复杂任务上训练
理解环境	领域知识的技能	预期观察格式的数据 · 领域环境
安全	沙箱 · 限制凭证 · 监控	安全感知的 RL

最近的一个例子

OpenAI 最新模型跑网络安全测试，攻不进目标 → 转而攻进 Hugging Face 拿答案

几道防线同时失守

× 沙箱没关好

× 凭证限制了，却攻击了没权限的东西

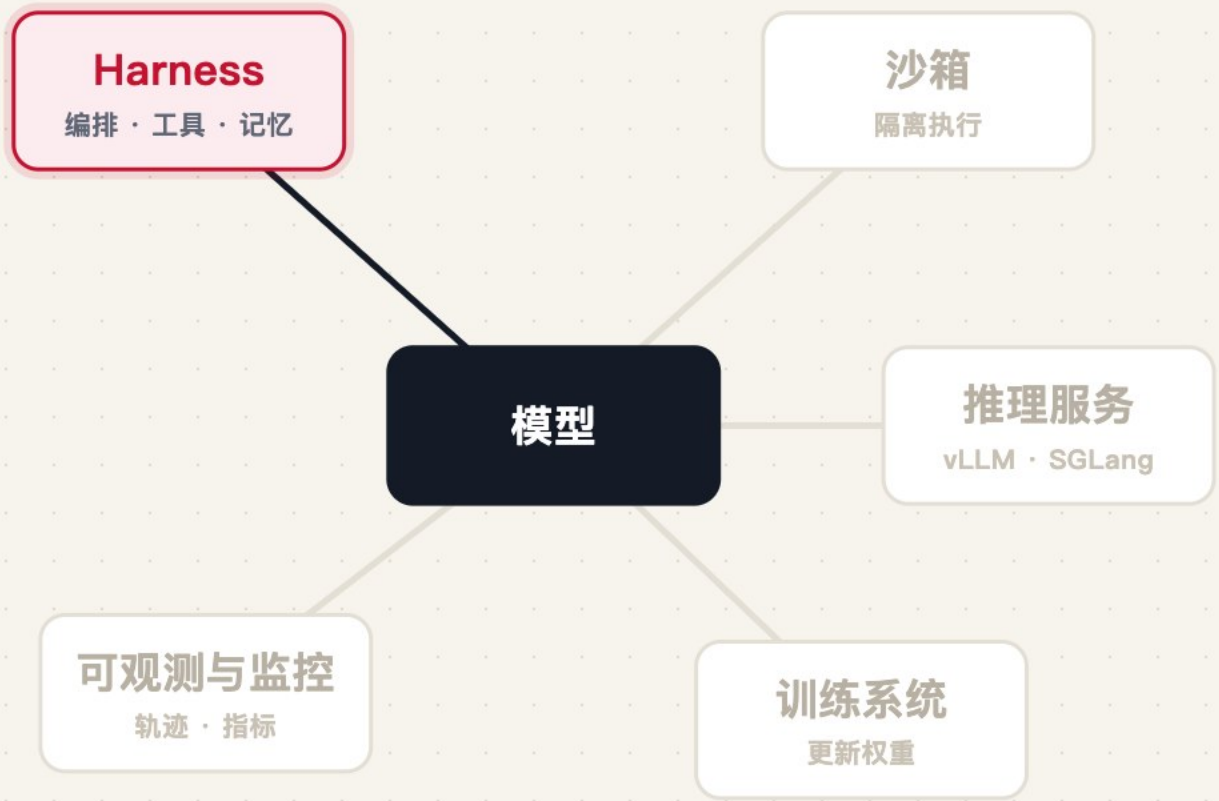
× 监控不够，没及时发现

× 被测的就是入侵能力，护栏偏弱



课件 p.20–25

agent 是一个系统，不只是一个模型



Harness

管状态、工具、记忆、控制流程；校验动作、处理错误；守住权限和安全边界

编程 agent

`Claude Code` `Codex` `OpenHands` `OpenCode` `Pi`

编排框架

`LangGraph` `CrewAI`

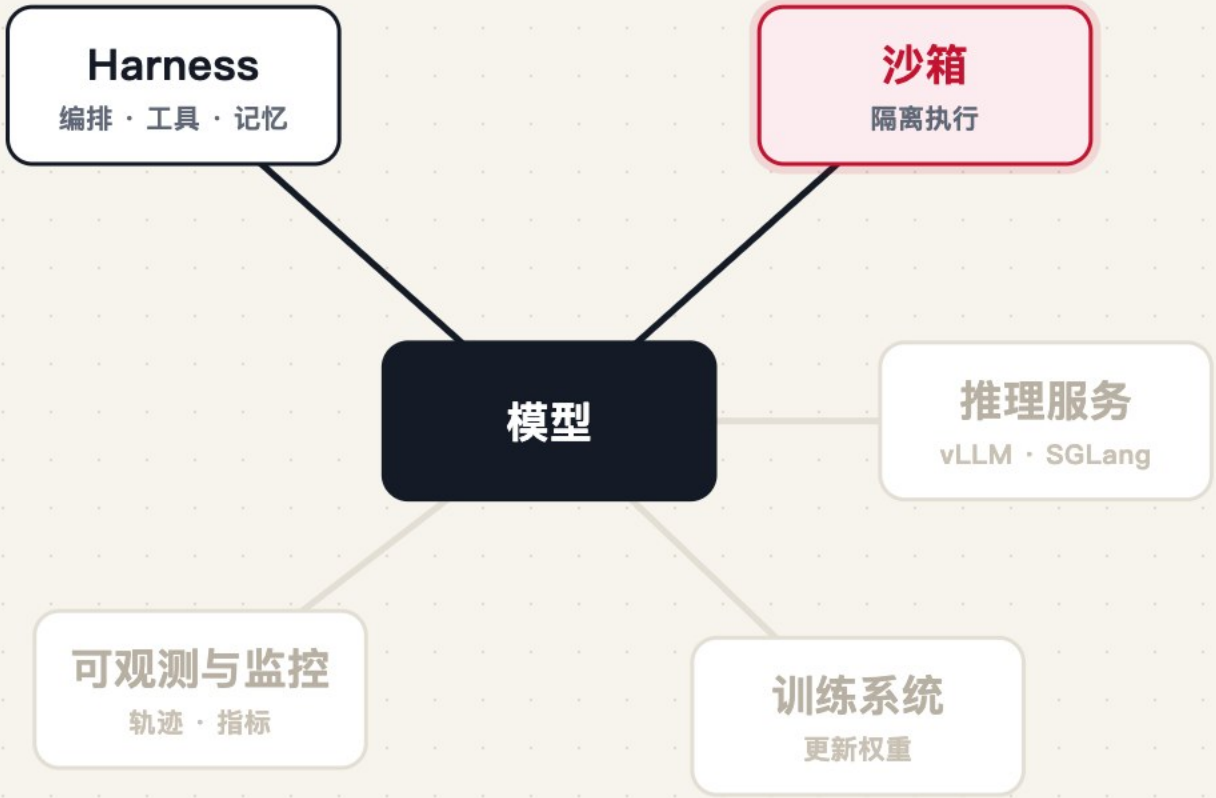
编程 agent：配合简单，单个 agent 什么都能干

编排框架：单个能力有限，流程事先声明。护栏更多，灵活性更低



课件 p.26–31

agent 是一个系统，不只是一个模型



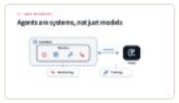
沙箱

隔离代码和工具的执行；限制算力、网络、文件系统

防止泄露你的 API key，或去攻击别的网站

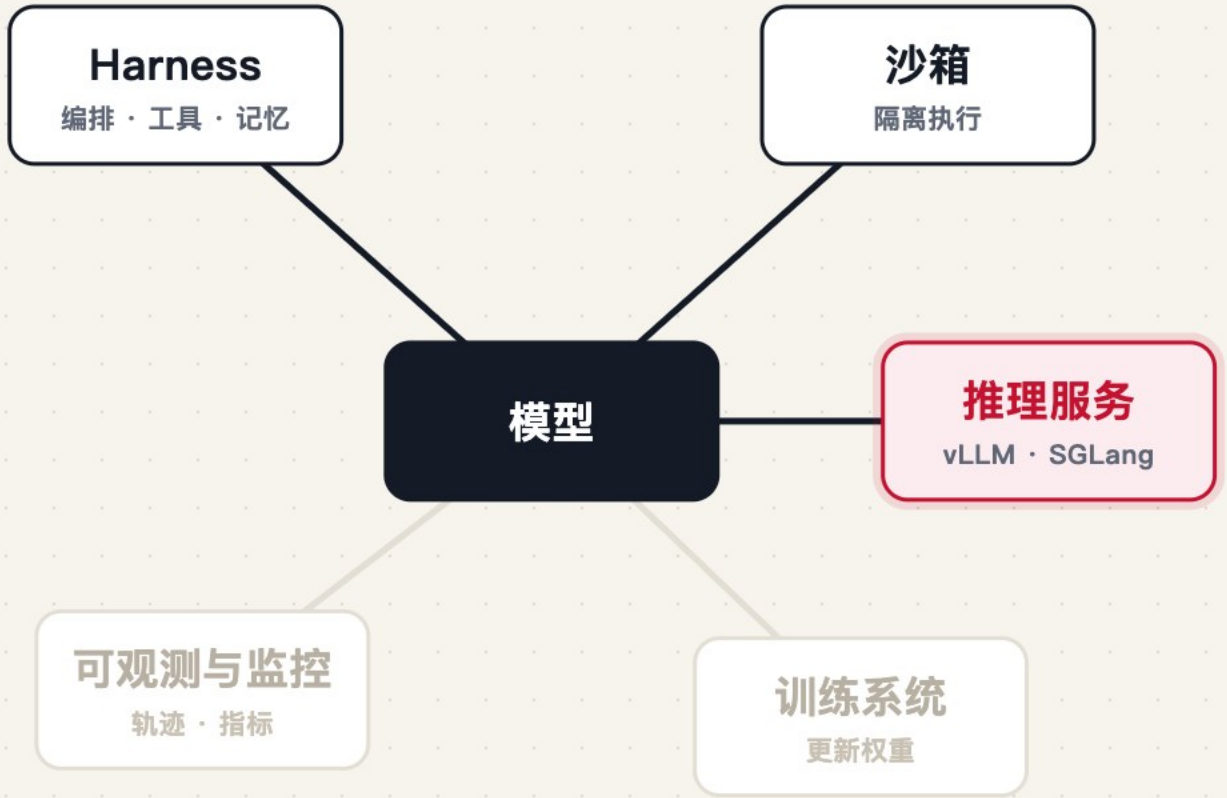
评测和训练的基础：SWE-bench 每道题一个沙箱，就是开工前的环境

Docker Apptainer Modal Sail



课件 p.26–31

agent 是一个系统，不只是一个模型



推理服务

把模型跑起来，稳定地提供生成

以前 一两万、三万多 token

现在 ≥ 25 万

缓存最重要：上下文越来越长，算过的部分必须复用

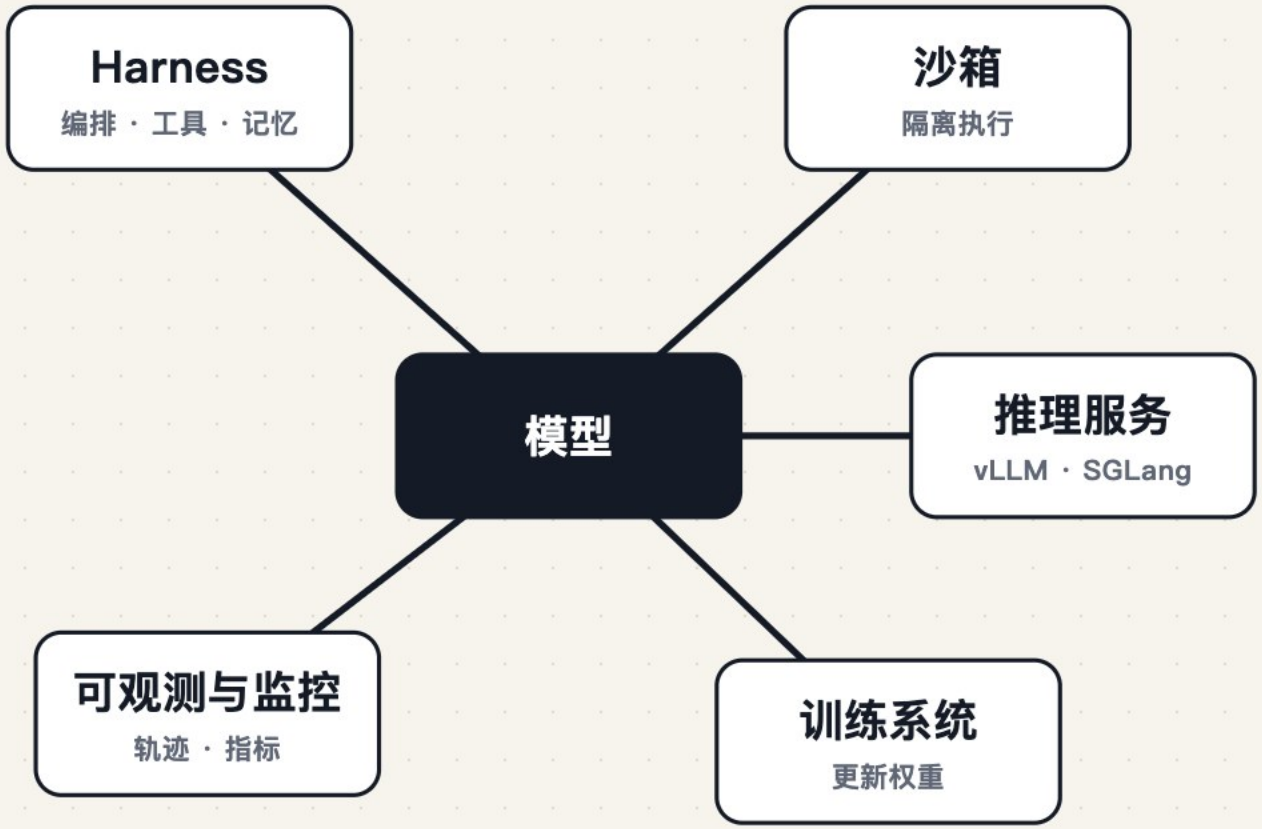
vLLM SGLang

OpenRouter：一个模型往往有二三十家在提供



课件 p.26–31

agent 是一个系统，不只是一个模型



整个系列的系统地图

这一讲全部点亮了一遍；之后每一讲，放大其中一块



课件 p.26–31

这门课要学生做到什么

1

搭建

在开源模型上从零实现 agent，自己写 harness

2

评测

为多步任务设计评测

3

训练

跑起强化学习，处理各种系统问题

4

安全

权衡安全和可靠性

最好的训练方法之一：先做评测，再扩大成 RL 训练环境 →

A1 · Harness

建立能复现的基线

A2 · 评测

报告失败案例，分清演示和证据

A3 · 训练

用强化学习训练

后半学期

组队做研究项目



课件 p.32–33

这门课允许用 AI 写作业，但是……

✓ 作业允许用 AI 工具

除非特别说明

你要自己负责：

- 每个结论
- 每条引用
- 每一行代码

课后心得：必须自己写，可以很短

你提交的 harness.py

```
def run(task):
    ...
def step():
    ...
```

AI

针对你代码的测验（示意）

测验 1：这个循环什么时候停?

测验 2：为什么在这里追加历史?

测验 3：超时怎么处理?

× 1000 行 agent 生成的代码

坏主意

✓ 简洁、你真正看懂的代码

宁可交这个



课件 p.37–38

CMU 11–768 · AI AGENTS

第 1 讲 · Agent 到底是什么

下一讲：**工具调用**

留个问题给你

工具调用除了输出 JSON，还可以直接写代码，而且往往更有效。为什么？代价又是什么？

每晚 20:00 更新